

Оглавление

- 1 *Кибизов К.О., Сальников А.Н.* Технические аспекты выбора пути передачи данных через физическую среду в кластере с графическими процессорами 4

Кибизов К.О., Сальников А.Н.

ТЕХНИЧЕСКИЕ АСПЕКТЫ ВЫБОРА ПУТИ ПЕРЕДАЧИ ДАННЫХ ЧЕРЕЗ ФИЗИЧЕСКУЮ СРЕДУ В КЛАСТЕРЕ С ГРАФИЧЕСКИМИ ПРОЦЕССОРАМИ

Введение

В вычислительном кластере данные между графическими процессорами (GPU) могут передаваться через разные физические среды: одни соединяют устройства внутри узла, другие — узлы между собой. Поэтому между двумя GPU обычно существует несколько возможных путей передачи. Например, передача данных может осуществляться как напрямую между устройствами, так и с промежуточным копированием через оперативную память. Такие варианты передачи возможны как между GPU в пределах одного вычислительного узла, так и между GPU и сетевым адаптером при межузловой передаче.

Выбор пути зависит не только от аппаратуры, но и от программного стека: операционной системы, драйверов устройств, коммуникационных библиотек и протоколов передачи. Программа взаимодействует с этим стеком через соответствующие интерфейсы: CUDA или OpenCL используются для работы с GPU, а MPI — для передачи данных между процессами. Решения, от которых зависит путь, принимаются на разных уровнях стека, поэтому в сложной среде путь не очевиден даже при известной топологии. От выбранного пути зависит задержка передачи, то есть время, необходимое для доставки данных от одного GPU к другому.

Для разных пар компонентов среды передачи данных задержка может заметно различаться. Неоднородность задержек между парами узлов кластера показана в [1], а в [2] предложен метод их измерения для GPU, расположенных на разных узлах. Настоящая работа продолжает этот подход и расширяет набор тестов проекта Clustbench в ветке `gpu_tests` [3]. Ранее на каждом узле запускался один MPI-процесс, обслуживавший все его GPU: передача внутри узла измерялась средствами CUDA без участия MPI, а между узлами всегда шла через оперативную память. Теперь каждому GPU соответствует отдельный MPI-процесс, поэтому любая передача, как внутри узла, так и между узлами, измеряется через

MPI, а при необходимости передачу можно принудительно вести через оперативную память.

Цель работы — выяснить, как настройки Open MPI, транспортного слоя UCX и планировщика Slurm определяют путь передачи данных между GPU, и исследовать задержки передачи в зависимости от размера сообщения, взаимного расположения GPU и среды передачи.

1. Модель коммуникационной среды и постановка задачи

Коммуникационная среда представляется в виде графа. Его вершины соответствуют аппаратным компонентам: процессорам, GPU, сетевым адаптерам и коммутаторам. Рёбра соответствуют физическим соединениям: PCIe, NVLink, межпроцессорному интерфейсу, Ethernet, InfiniBand. При тестировании GPU передают друг другу сообщения — синтетически сформированными блоками данных заданного размера. При передаче сообщение проходит по графу через последовательность вершин и рёбер, то есть по некоторому пути. Множество возможных путей зависит от топологии и программного окружения, а конкретный путь выбирает транспортный слой. Пример такой среды — узел с несколькими GPU — показан на рис. 1.

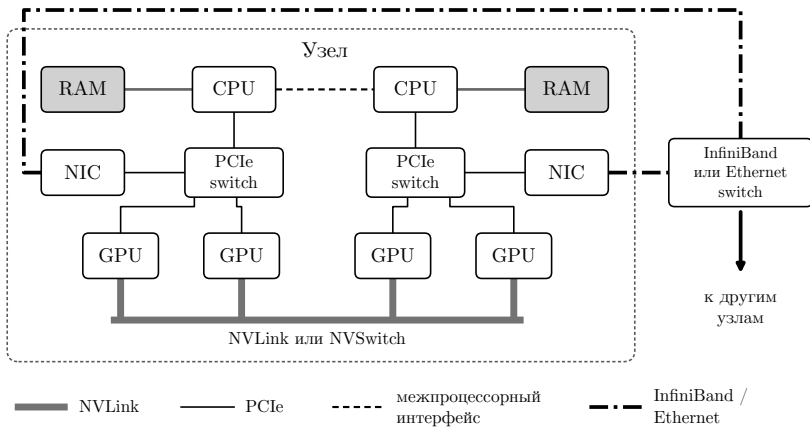


Рис. 1: Пример архитектуры узла с несколькими GPU (CPU — процессор, RAM — оперативная память, NIC — сетевой адаптер, switch — коммутатор)

Основные пути передачи данных между GPU показаны на рис. 2. Без промежуточного копирования данные передаются внутри узла средствами GPUDirect Peer-to-Peer [4] (путь а), между узлами — GPUDirect RDMA [5] (путь с). В остальных случаях данные копируются через оперативную память (пути b и d).

Предметом изучения служит сквозная задержка передачи сообщения между упорядоченной парой различных устройств, то есть время от начала передачи на устройстве-отправителе до завершения приёма на устройстве-получателе.

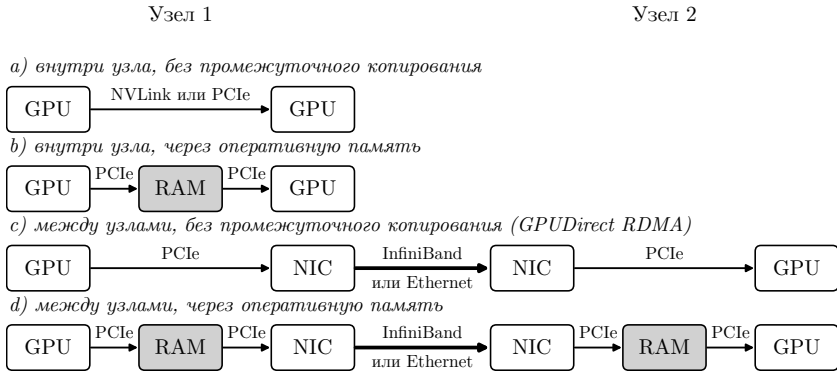


Рис. 2: Пути передачи данных между GPU (RAM — оперативная память, NIC — сетевой адаптер)

2. Настройка коммуникационного стека

Путь передачи задаётся на нескольких уровнях: аппаратуры, драйверов, программного обеспечения и самой программы. Ниже настройки рассмотрены по этим уровням.

2.1 Уровень аппаратуры

На уровне аппаратуры путь определяется расположением устройств в узле: к какому процессору и коммутатору PCIe подключены GPU и сетевые адаптеры и соединены ли GPU через NVLink. Топологию узла показывает команда `nvidia-smi topo -m`.

При передаче без промежуточного копирования данные идут между GPU и сетевым адаптером мимо процессора, поэтому адаптер нужно выбирать по близости к GPU в топологии PCIe, а не к ядрам процесса, как это по умолчанию делает UCX.

2.2 Уровень драйверов

Прямой доступ GPU к памяти другого GPU того же узла (GPUDirect Peer-to-Peer [4]) обеспечивает драйвер NVIDIA. Разрешён ли он для пары GPU, показывает команда `nvidia-smi topo -p2p`. Для передачи между узлами без промежуточного копирования (GPUDirect RDMA [5]) сетевой адаптер должен обращаться к памяти GPU, для чего должен быть загружен модуль ядра `nvidia_peermem`. На рассмотренных узлах оба условия выполнены.

2.3 Уровень программного обеспечения

Архитектура UCX. Open MPI [6] передаёт сообщения через транспортный слой UCX [7]. UCX состоит из двух уровней. Нижний уровень предоставляет транспорты — способы передачи для конкретной аппаратуры (таблица 1). Верхний уровень реализует протоколы передачи сообщений и для каждой пары процессов выбирает, какими транспортом пользоваться.

Параметры сборки. UCX 1.15.0 собран с поддержкой CUDA, InfiniBand и CMA, но без библиотеки GDRCopy, ускоряющей копирование небольших сообщений из памяти GPU. Open MPI 5.0.10 собран с этой версией UCX и с поддержкой CUDA. Использовался CUDA Toolkit 12.8.

Параметры Open MPI и UCX. В обеих средах передачи, `auto` и `host` (раздел 3), Open MPI передаёт сообщения через UCX (`OMPI_MCA_pml=ucx`) с включённой поддержкой CUDA (`OMPI_MCA_opal_cuda_support=1`). Набор доступных транспортов задаётся переменной `UCX_TLS`. В среде `auto` в него входят транспорты CUDA, а переменная `UCX_IB_GPU_DIRECT_RDMA=y` разрешает сетевому адаптеру обращаться к памяти GPU. В среде `host` в MPI передаются только указатели на оперативную память, поэтому транспорты CUDA исключены, а GPUDirect RDMA запрещена. Если нужный транспорт не указан или недоступен, UCX не сообщает об ошибке, а передаёт данные другим путём, как правило через оперативную память.

Транспорт	Назначение	auto	host
<code>cuda_copy</code>	копирование между памятью GPU и оперативной памятью	+	—
<code>cuda_ipc</code>	доступ к памяти GPU другого процесса узла (GPUDirect P2P)	+	—
<code>rc, ud, dc</code>	передача между узлами по сети InfiniBand	+	+
<code>cma</code>	копирование между процессами узла средствами ядра ОС	+	+
<code>sm</code>	передача через разделяемую память узла	+	+
<code>self</code>	передача внутри процесса	+	+

Таблица 1: Транспорты UCX в средах `auto` и `host`

Сообщение передаётся одним из двух протоколов. В протоколе `eager` отправитель сразу посылает данные, не дожидаясь готовности получателя, и получатель при необходимости складывает их во временный буфер. Сообщение в памяти GPU при этом копируется через оперативную память (пути `b` и `d` на рис. 2). В протоколе `rendezvous` отправитель сначала посылает только запрос, и данные передаются после того, как получатель подготовит буфер приёма. Это требует дополнительного обмена служебными сообщениями, но позволяет передать данные без промежуточного копирования (пути `a` и `c`). Обычно протокол `eager` выгоден для небольших сообщений, поскольку не требует согласования, а `rendezvous` — для крупных. Однако для сообщений в памяти GPU `eager` требует копирования через оперативную память, и его преимущество может теряться (раздел 5.1).

Выбор протокола определяется порогом `UCX_RNDV_THRESH`, который задаётся при запуске: сообщения меньше порога передаются протоколом `eager`, остальные — протоколом `rendezvous`. Использовались три значения порога. Значение по умолчанию UCX выбирает автоматически. При значении 0 любое сообщение передаётся протоколом `rendezvous`. Третье значение выбрано больше наибольшего размера сообщения в измерениях, поэтому все сообщения передаются протоколом `eager`. Два крайних значения позволяют сравнить оба протокола на всём диапазоне размеров.

Планировщик Slurm. Процессы запускаются командой `srun` с интерфейсом `PMIx`, по одному на GPU. Флаг `--gres-flags=enforce-binding` требует, чтобы ядра процесса находились на том же сокете, что и его GPU [8], иначе передача данных между процессором и GPU пойдёт через межпроцессорный интерфейс. Ядра закрепляются за процессом параметром

`--cpu-bind=cores.` GPU запрашиваются на узел (`--gpus-per-node`), а не на процесс: иначе процесс видит только свой GPU, и транспорт CUDA IPC не работает. Вариант `--gpu-bind=closest` не используется, поскольку может выдать нескольким процессам один и тот же GPU.

2.4 Уровень программы

На уровне самой программы важен порядок вызовов. UCX читает свои параметры, в том числе сетевой адаптер (`UCX_NET_DEVICES`), при инициализации внутри вызова `MPI_Init`, поэтому они должны быть заданы до него. Адаптер зависит от GPU процесса, а номер процесса внутри узла, по которому назначается GPU, до `MPI_Init` доступен только из переменных окружения системы запуска (`SLURM_LOCALID`).

3. Способ измерения

Задержка измеряется на стороне получателя таймером `MPI_Wtime()`. Для упорядоченной пары GPU (i, j) в k -м замере получатель фиксирует момент t_k^{rdy} готовности принять данные и момент t_k^{end} завершения приёма, а задержка оценивается как

$$t_{ij,k} = t_k^{\text{end}} - t_k^{\text{rdy}}.$$

Завершение приёма имеет однозначный смысл: к этому моменту данные находятся в буфере получателя. Начальная отметка может предшествовать началу передачи, поэтому $t_{ij,k}$ является оценкой сверху для сквозной задержки. Зато обе отметки снимаются одним процессом, и согласование таймеров разных узлов не требуется.

Каждому GPU соответствует отдельный MPI-процесс. Open MPI используется в режиме CUDA-aware, в котором вызовы MPI принимают указатели на память GPU.

Среды передачи. В среде `auto` указатели на буферы в памяти GPU передаются непосредственно в вызовы MPI, а путь передачи выбирает UCX. После приёма выполняется синхронизация устройства, поскольку транспорт CUDA IPC может сообщить о завершении приёма раньше, чем копирование в память GPU фактически завершится. В среде `host` программа сама копирует данные в оперативную память и передаёт их средствами MPI, а для пар внутри одного узла использует общий буфер в разделяемой памяти узла.

Режимы тестирования. В режиме `one_to_one` пары GPU перебираются последовательно, и в каждый момент времени передача идёт только между одной парой. В режиме `all_to_all` передачи между всеми парами идут одновременно. Начальная отметка у получателя в этом режиме общая для всех источников, а конечная фиксируется отдельно для каждого источника.

4. Экспериментальная среда и методика измерений

Измерения выполнены на вычислительном комплексе ЦКП «Информатика» [9] на узлах двух типов с GPU NVIDIA A100 40 ГБ. Узлы xFusion G5500 V7 содержат по четыре GPU, подключённых по шине PCIe, и по два процессора Intel Xeon Gold 6444Y. Узлы xFusion G8600 V7 содержат по восемь GPU в исполнении SXM4, объединённых коммутаторами NVSwitch, и по два процессора Intel Xeon Platinum 8462Y+. Узлы соединены сетью InfiniBand 100 Гбит/с.

В измерениях использовались по два GPU на двух узлах G5500 V7 и четыре GPU на одном узле G8600 V7. Таким образом, рассматриваются упорядоченные пары GPU трёх типов: 12 пар внутри узла G8600 V7 (NVLink), 4 пары внутри узлов G5500 V7 и 8 пар на разных узлах G5500 V7 (PCIe и InfiniBand). Теоретическая пропускная способность NVLink третьего поколения, установленного на A100, составляет 300 ГБ/с в одном направлении, а сети InfiniBand 100 Гбит/с — 12.5 ГБ/с.

Модуль сам выставляет переменные UCX, от которых зависит путь передачи: набор транспортов, разрешение GPUDirect RDMA и сетевой адаптер (например, `mlx5_0:1`). Адаптер выбирается среди адаптеров InfiniBand по наиболее длинному общему префиксу пути в дереве устройств PCIe с путём к GPU, то есть за общим с ним коммутатором PCIe. Значения переменных записываются в заголовок каждого файла результатов. Переменная выставляется, только если она не задана в окружении, поэтому скрипт запуска перед каждым запуском сбрасывает значения, унаследованные из окружения пользователя.

Размер сообщения S изменялся от 1 КБ до 16 МБ, и для каждой пары GPU и каждого размера выполнялась серия замеров. Результаты приводятся для режима `one_to_one`, в котором передачи разных пар не конкурируют между собой и задержка определяется путём передачи.

Первые передачи в каждой серии не учитываются: так из результата исключаются возможные разовые издержки на установление соединений транспортного слоя и регистрацию буферов. В замерах внутри серии могут наблюдаться периодические всплески задержки [10], поэтому типичное значение для пары оценивается не средним, а медианой $\tilde{t}_{ij}$, устойчивой к выбросам [11]. Для каждого типа пар и размера сообщения приводится медиана по парам $\tilde{t} = \text{med}_{(i,j)} \tilde{t}_{ij}$, а эффективная скорость передачи оценивается как $B = S/\tilde{t}$.

5. Результаты

5.1 Влияние порога протокола UCX

Поскольку протокол выбирается по размеру сообщения, в среде `auto` от размера зависит и путь передачи. Порог по умолчанию различается между группами узлов: на узле G8600 V7 он лежит между 8 и 16 КБ, на узлах G5500 V7 — между 256 и 512 КБ. Ниже порога задержку ограничивает копирование через оперативную память, и при переходе через порог она резко падает (рис. 3).

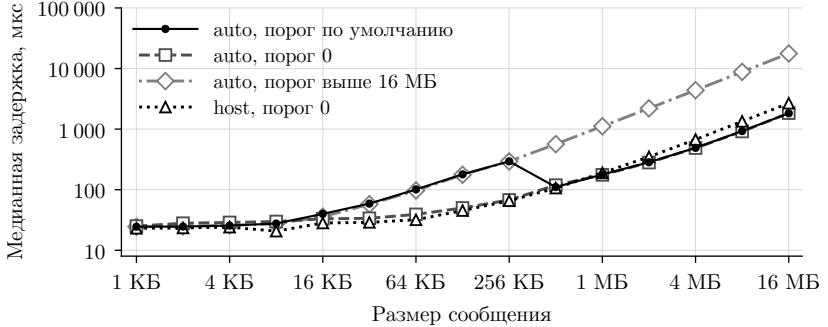


Рис. 3: Задержка для пар GPU на разных узлах G5500 V7 (PCIe + InfiniBand) в среде `auto` при трёх значениях `UCX_RNDV_THRESH` и в среде `host` при `UCX_RNDV_THRESH=0`, режим `one_to_one`

В среде `host` порог на путь передачи не влияет: данные в любом случае проходят через оперативную память, и задержки при разных значениях порога различаются мало. Поэтому среда `host` далее сравнивается со средой `auto` при одном и том же пороге.

Если порог больше любого сообщения, все сообщения передаются протоколом `eager`, и для крупных сообщений передача оказывается в несколько раз медленнее, чем даже в среде `host`.

Поэтому основные измерения выполнены с `UCX_RNDV_THRESH=0`, при котором любое сообщение передаётся протоколом `rendezvous` и путь передачи не зависит от размера сообщения. Для небольших сообщений это почти не увеличивает задержку, а для сообщений среднего размера на узлах G5500 V7 заметно её снижает.

5.2 Задержка и скорость передачи

Медианные задержки при `UCX_RNDV_THRESH=0` для сообщений 1 КБ и 16 МБ приведены в таблице 2.

Для небольших сообщений задержка во всех случаях составляет десятки микросекунд, и заметно меньше она только в среде `auto` внутри узла G8600 V7. Для крупных сообщений задержку определяет скорость передачи по пути. В среде `auto` передача внутри узла G8600 V7 достигает около 75% теоретической пропускной способности NVLink, а между узлами G5500 V7 — около 70% пропускной способности InfiniBand. Задержки отдельных пар одного типа различаются лишь на несколько процентов, то есть в режиме `one_to_one` неоднородность задержек определяется типом пары, а не отдельными парами.

Пары	Среда	$\bar{t}$, мкс		B , ГБ/с
		1 КБ	16 МБ	16 МБ
внутри G8600	<code>auto</code>	12.8	72.0	228
	<code>host</code>	18.1	1258	13.0
внутри G5500	<code>auto</code>	23.4	834	19.6
	<code>host</code>	22.7	1253	13.1
между G5500	<code>auto</code>	25.0	1839	8.9
	<code>host</code>	23.1	2673	6.1

Таблица 2: Медианная задержка и эффективная скорость передачи для пар внутри узла G8600 V7, внутри узла G5500 V7 и на разных узлах G5500 V7

Для пар внутри узла G5500 V7 скорость составляет лишь около 20 ГБ/с: по таблице выбора протоколов UCX (`UCX_PROTO_INFO=y`) в задании на двух узлах данные между GPU одного узла идут через сетевой адаптер. В задании на одном узле для тех же GPU используется транспорт `cuda_ipc`, и 16 МБ передаются за 72 мкс, более чем в 10 раз быстрее. Как показали сообщения Open MPI, в задании на нескольких узлах процессы получают через систему запуска только сетевую часть адреса UCX друг друга, в том числе соседей по узлу, и другие транспорты UCX для них недоступны.

В среде `host` задержка для пар внутри узла одинакова на обеих

группах узлов, то есть NVLink в этой среде не используется. Оценка по разности задержек для пар на разных узлах и внутри узла показывает, что сама передача по сети использует InfiniBand более чем на 90%, то есть заметно лучше, чем GPUDirect RDMA.

5.3 Выигрыш от передачи без промежуточного копирования

На рис. 4 показано, во сколько раз передача в среде `auto` быстрее, чем в среде `host`. Внутри узла G8600 V7 среда `auto` выигрывает при любом размере сообщения, а для крупных сообщений — примерно в 17 раз.

На узлах G5500 V7 выигрыш появляется только для крупных сообщений: от 1 МБ между узлами и от 4 МБ внутри узла. Он не превышает 1.5 раза. Для пар внутри узла это объясняется тем, что данные идут через сетевой адаптер (раздел 5.2): у такого пути большая постоянная задержка, и для сообщений среднего размера среда `auto` оказывается даже медленнее среды `host`. Таким образом, передача без промежуточного копирования выгодна не всегда.

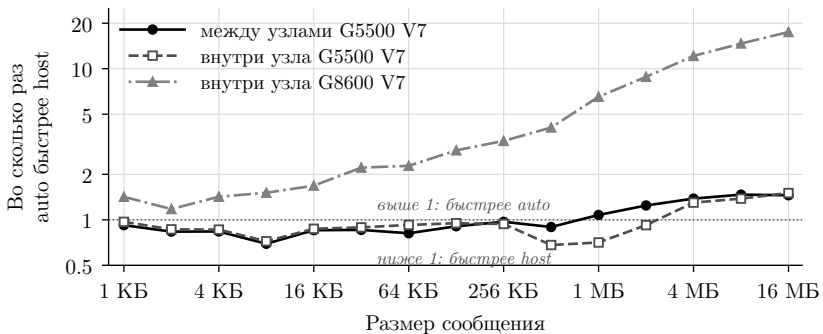


Рис. 4: Во сколько раз передача в среде `auto` быстрее, чем в среде `host` (отношение задержек `host/auto`), при `UCX_RNDV_THRESH=0`, режим `one_to_one`

Заключение

Разработан модуль, измеряющий задержки передачи данных между всеми упорядоченными парами GPU кластера. Путь передачи определяется не только аппаратурой, но и настройками Open MPI, UCX и Slurm, причём при неподходящих настройках UCX не сообщает об ошибке, а передаёт данные через оперативную память. Поэтому параметры UCX, включая порог протокола rendezvous, следует задавать явно и сохранять вместе с результатами, сетевой адаптер выбирать по близости к GPU, а ядра процессора выделять на сокетe GPU.

Измерения показали, что выигрыш от передачи без промежуточного копирования зависит от расположения GPU и от пути, выбранного UCX: внутри узла G8600 V7 он наблюдается при любом размере сообщения, а между узлами G5500 V7 — только для крупных сообщений. В задании на нескольких узлах UCX может передавать данные между GPU одного узла через сетевой адаптер, что замедляет передачу более чем в 10 раз. Поэтому выбранный UCX путь следует проверять, например с помощью UCX_PROTO_INFO.

В дальнейшем планируется исследовать режим `all_to_all` и выделить группы пар GPU со сходными задержками аналогично [1] на большем числе узлов.

Литература

1. Сальников А. Н., Бегаев А. А., Майсурадзе А. И. *Кластеризация задержек между узлами суперкомпьютера и визуализация полученных результатов* // Современные проблемы математического моделирования, обработки изображений и параллельных вычислений 2017: труды Международной научной конференции (пос. Дивноморское, 4–11 сентября 2017 г.). Ростов-на-Дону: ДГТУ-Принт, 2017. Т. 1. С. 206–215.
2. Бегаев А. А., Сальников А. Н. *Метод измерения задержек при передаче данных между графическими процессорами, находящимися на разных узлах вычислительного кластера* // Вестник Московского университета. Серия 15: Вычислительная математика и кибернетика. 2020. Вып. 1. С. 3–13.

3. *Clustbench / network-tests2, вемка gpu_tests*. https://github.com/clustbench/network-tests2/tree/gpu_tests (дата обращения: 28.09.2026).
4. NVIDIA. *CUDA C++ Programming Guide: Peer Device Memory Access*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (дата обращения: 20.09.2026).
5. NVIDIA. *GPUDirect RDMA*. <https://docs.nvidia.com/cuda/gpudirect-rdma/> (дата обращения: 20.09.2026).
6. *Open MPI 5.0.x Documentation: CUDA*. <https://docs.openmpi.org/en/v5.0.x/tuning-apps/networking/cuda.html> (дата обращения: 20.09.2026).
7. *Unified Communication X*. <https://openucx.org/> (дата обращения: 20.09.2026).
8. *Slurm Workload Manager: Generic Resource (GRES) Scheduling*. <https://slurm.schedmd.com/gres.html> (дата обращения: 20.09.2026).
9. Центр коллективного пользования «Информатика» ФИЦ ИУ РАН: *Вычислительная инфраструктура*. <https://docs.hpc.frcsc.ru/hardware/> (дата обращения: 20.09.2026).
10. Волчанинов А. П., Сальников А. Н. *Исследование структуры серии измерений задержки при нагрузочном тестировании коммутационной среды вычислительного кластера* // Суперкомпьютерные дни в России: труды международной конференции (Москва, 25–26 сентября 2023 г.). М.: МАКС Пресс, 2023. С. 42–52.
11. Hoeffler T., Belli R. *Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results* // Proceedings of SC'15. 2015. Art. 73. P. 1–12. DOI: 10.1145/2807591.2807644.